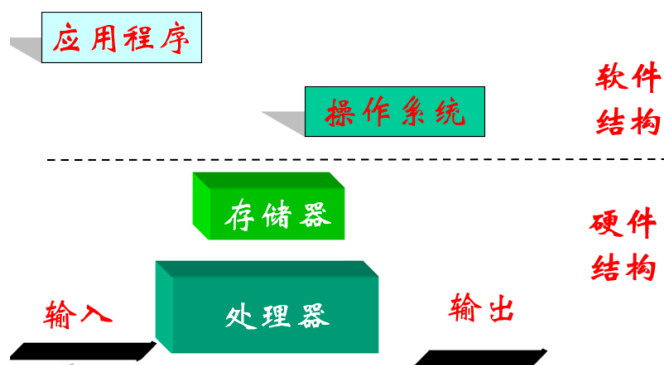
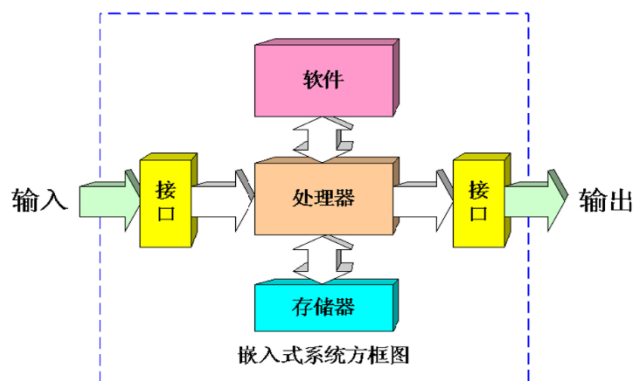


第 14 讲 案例分析

嵌入式系统概念：以应用为中心、以计算机技术为基础，软硬件可裁剪，适应应用系统对功能、可靠性、成本、体积、功耗严格要求的专用计算机系统

➤ 嵌入式系统主要由以下四部分组成：

- ☐ 嵌入式处理器
- ☐ 外围硬件设备
- ☐ 嵌入式操作系统
- ☐ 用户应用程序



一、 嵌入式系统设计分析

➤ 嵌入式系统设计分析思路

- 详细分析嵌入式系统的需求
- 从需求引出系统设计思路
- 给出硬件设计和软件设计方法
 - ☐ 结构设计
 - ☐ 接口设计
 - ☐ 程序设计（驱动程序、应用程序等）

➤ 案例分析——定位系统

➤ 硬件分配需求

- ☐ 定位

主动定位：待定位设备主动发出定位信号，用于定位系统对该设备进行侦测定位。

被动定位：待定位设备根据定位系统发出的定位信号进行自我定位。

主动



定位信号选择
光波？声波？电磁波？



定位系统构建
无线传感网络？蜂窝网络？

- ☐ 定位



主动定位？
被动定位？☒

- 利用北斗定位系统进行被动定位

被动



定位信号选择
GPS？北斗？特殊定位信号？

室内：蓝牙/声波（当蓝牙设备很多时）

北斗：经济实用

□ 通信



定位结果如何上报
GSM? GPRS? 3G? 4G?

远距离传输（蓝牙为短距离）

➤ 需求总结 1

- 采用北斗进行定位，利用 GSM 网络定期短信上报定位结果

➤ 硬件设计方案 1

- 北斗定位模块（能定位就行）
- GSM 模块（能通信就行，不需要多复杂的系统）
- 单片机



✓ 定位设备上是否需要存储所有定位结果？

□ 特性需求

✓ 定位设备是否支持定位结果导出功能？

✓ 定位设备是否支持路线偏离预警？

➤ 需求总结 2

- 采用北斗定位进行定位，利用 GSM 网络定期（2G 网络）短信上报定位结果
- 可以一次性导出所有存储的定位结果
- 可以配置预设路线或区域，进行定位偏离预警

➤ 硬件设计方案 2

- 北斗定位模块
- GSM 模块
- ARM（为了能外部扩展）
- NAND/NOR Flash
- 串口/USB（为增加外部链接，二者通信速率不一样，USB 更高）

□ 人机交互



✓ PC机无串口接口，串口配置不方便

➤ 需求总结 3

- 定位、上报
- 配置导入导出、偏离预警
- 近距离通信

✓ 利用手机进行近距离通信，实现配置/导出

➤ 硬件设计方案 3

- 北斗定位模块
- GSM 模块
- ARM
- NAND/NOR Flash
- 蓝牙/WIFI（更换掉串口和 USE——但在实际应用中也许会保留从而方便调试）

□ 特色需求

实时跟踪时短信告警太频繁



✓ 一键紧急告警

✓ 实时跟踪（定位周期小于1秒）

➤ 需求总结 4

- 定位、上报
- 配置导入导出、偏离预警
- 近距离通信
- 一键告警、实时监控

➤ 硬件设计方案 4

- 北斗定位模块
- 3G/4G（更换 GSM）：实现任意通信

- ARM
- NAND/NOR Flash
- 蓝牙/WIFI（保留近距离通信）

□ 特色需求



- ✓ 双向语音对讲
- ✓ 偏离预设路线或范围时震动提示、红灯提示

➤ 需求总结5

- 定位监控
- 电子围栏
- 紧急报警
- 双向语音
- 提示功能

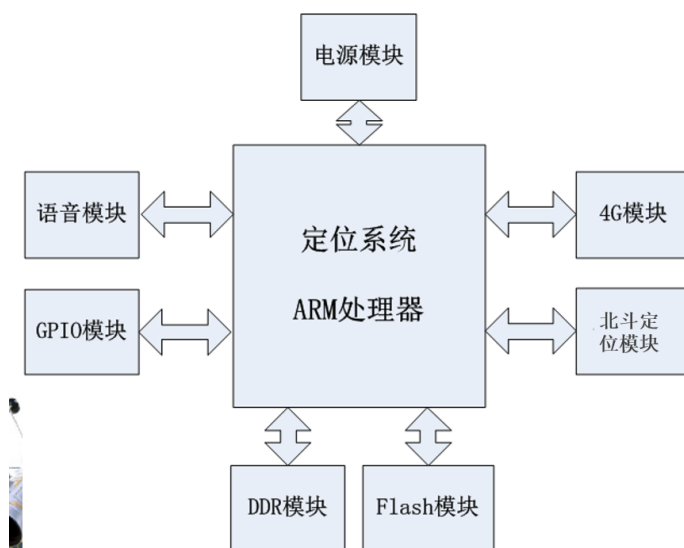
➤ 硬件设计方案5

- 北斗定位模块
- 3G/4G/蓝牙/WIFI
- ARM
- NAND/NOR Flash
- 语音采集、播放
- 振动马达
- LED指示灯

➤ 硬件设计总结

- | | |
|------------------|-----|
| □ 北斗定位模块 | 外设 |
| □ 3G/4G/蓝牙/WIFI | 网络 |
| □ ARM | CPU |
| □ NAND/NOR Flash | 存储 |
| □ 语音采集、播放 | I/O |
| □ 振动马达 | 外设 |
| □ LED指示灯 | I/O |

二、 硬件系统设计



三、 核心模块设计

- 按照**模块化**进行系统任务分类
- 涉及模块如何划分、模块间如何进行交互？
 - 自底向上的进行模块划分（驱动、库、应用）
 - 按照业务流进行模块划分（发现公共模块、每个模块用进程来实现）

公共模块：参数管理模块

通信：Unix socket

- 模块划分

➤ 按层次划分：

- ❑ 嵌入式操作系统（U-Boot、OS）
- ❑ 驱动程序
 - 网络（4G）驱动
 - 北斗定位模块驱动
 - 音频驱动
 - Flash 驱动
 - GPIO 驱动
 - 其它总线驱动（I2C/SPI/USB）
- ❑ 中间件库
 - 北斗定位数据读取
 - Flash 读写
 - IO 设备读写
- ❑ 应用程序

➤ 按业务划分

- ❑ 音频采集编码模块
- ❑ 音频解码播放模块
- ❑ 北斗定位数据读取模块
- ❑ 参数管理模块
- ❑ 定位业务模块
- ❑ IO 模块
- ❑ 对外交互模块
- ❑ 系统升级模块

➤ 层次化编程

➤ 总线与设备驱动

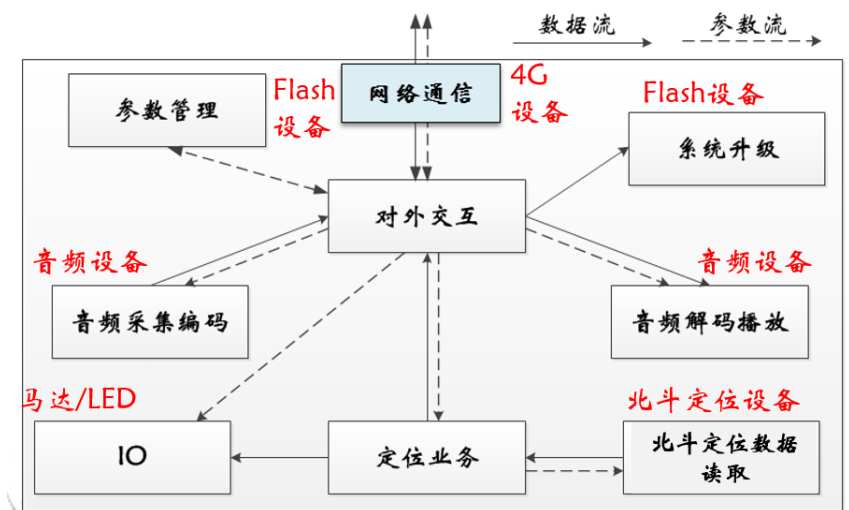
- ❑ GPIO 驱动与振动马达、LED 灯
- ❑ 串口驱动与北斗定位设备
- ❑ I2S 驱动与音频设备

➤ 中间件库与应用程序

- ❑ GPIO 库与 IO 模块
- ❑ Flash 库与系统升级、参数管理
- ❑ 音频采集播放库与音频采集、播放

➤ 模块化编程

- ❑ 对外交互（通过 4G 网络）
- ❑ 参数存储与读取
- ❑ 系统升级
- ❑ 音频采集-编码-发送
- ❑ 音频接收-解码-播放
- ❑ 定位业务
- ❑ 模块间交互



四、 软件设计思路归纳

➤ 总体设计过程

- 首先寻找实现目标系统的各种不同的方案（需求分析阶段得到的数据流图是设想各种可能方案的基础）。
- 然后从这些供选择的方案中选取若干个合理的方案，为每个合理的方案都准备一份系统流程图，列出组成系统的所有物理元素，进行成本效益/分析，并且制定实现出一个最佳方案的进度计划。
- 最后推荐最佳方案，用户接受后，为这个最佳方案设计软件结构。进行必要的数据库设计，确定测试要求并制定测试计划。

- ❑ 总体设计过程由两个主要阶段(包括 9 个步骤)组成:
 - (1)系统设计阶段, 确定系统的具体实现方案;
 - (2)结构设计阶段, 确定软件结构。
- ❑ 设计原理:模块化、抽象、信息隐藏和局部化、模块独立
- ❑ 模块的独立程度度量: 内聚和耦合, 在软件设计中应该追求尽可能松耦合、高内聚的系统。
- ❑ 启发规则: 7 条规则

➤ 设计过程

总体设计过程通常由两个主要阶段组成:

- (1)系统设计阶段, 确定系统的具体实现方案;
- (2)结构设计阶段, 确定软件结构。

典型的总体设计过程包括下述 9 个步骤:

- | | |
|------------|-----------|
| 1.设想供选择的方案 | 6. 设计数据库 |
| 2.选取合理的方案 | 7. 制定测试计划 |
| 3.推荐最佳方案 | 8. 书写文档 |
| 4.功能分解 | 9. 审查和复审 |
| 5.设计软件结构 | |

➤ 抽象

抽象就是抽出事物的本质特性而暂时不考虑它们的细节。

软件工程过程的每一步都是对软件解法的抽象层次的一次精化。在可行性研究阶段, 软件作为系统的一个完整部件; 在需求分析期间, 软件解法是使用在问题环境内熟悉的方式描述的; 当由总体设计向详细设计过渡时, 抽象的程度也就随之减少了; 最后, 当源程序写出来以后, 也就达到了抽象的最低层。

➤ 模块独立

模块独立化有如下优点:

- 1.有效的模块化(即具有独立的模块)的软件比较容易开发出来。
- 2.独立的模块比较容易测试和维护。

模块的独立程度可以由内聚和耦合两个定性标准度量:

耦合衡量不同模块彼此间互相依赖(连接)的紧密程度;

内聚衡量一个模块内部各个元素彼此结合的紧密程度。

➤ 耦合

- ❑ 耦合是对一个软件结构内不同模块之间互连程度的度量。耦合强弱取决于模块间接口的复杂程度, 进入或访问一个模块的点, 以及通过接口的数据。
- ❑ 在软件设计中应该追求尽可能松散耦合的系统。
- ❑ 模块间的耦合程度强烈影响系统的可理解性、可测试性、可靠性和可维护性。
- ❑ 如果两个模块中的每一个都能独立地工作而不需要另一个模块的存在, 那么它们彼此完全独立, 这意味着模块间无任何连接, 耦合程度最低。但是, 在一个软件系统中不可能所有模块之间都没有任何连接。

➤ 内聚

内聚标志一个模块内各个元素彼此结合的紧密程度, 简单地说, 理想内聚的模块只做一件事情。

设计时应该力求做到高内聚, 内聚和耦合是密切相关, 模块内的高内聚往往意味着模块间的松耦合。

➤ 启发规则

1. 改进软件结构提高模块独立性

设计出软件的初步结构以后, 应该审查分析这个结构, 通过**模块分解或合并, 力求降低耦合提高内聚**。
例如, 多个模块公有的一个子功能可以独立成一个模块, 由这些模块调用; 有时可以通过分解或合并模块以减少控制信息的传递及对全程数据的引用, 并且降低接口的复杂程度。

2. 模块规模应该适中

模块过大: 可理解程度下降

模块过小: 开销大于有效操作、接口复杂

经验表明, 一个模块的规模不应过大, 最好能写在一页纸内(通常不超过 60 行语句)。有人从心理学角度研究得知, 当一个模块包含的语句数超过 30 以后, 模块的可理解程度迅速下降。

过大的模块往往是由于分解不充分, 但是进一步分解必须符合问题结构, 一般说来, 分解后不应该降低模块独立性。

过小的模块开销大于有效操作, 而且模块数目过多将使系统接口复杂。因此过小的模块有时不值得单独存在, 特别是只有一个模块调用它时, 通常可以把它合并到上级模块中去而不必单独存在。

3. 深度、宽度、扇出和扇入都应适当

深度表示软件结构中控制的层数。

宽度是软件结构内同一个层次上的模块总数的最大值。

扇出是一个模块直接控制(调用)的模块数目。扇出太大, 应增加中间层次的控制模块; 扇出小时, 把下级模块进一步分解成若干个子功能模块或合并到它的上级模块中去。

扇入是表明有多少个上级模块直接调用它。

设计得很好的软件结构通常顶层扇出比较高, 中层扇出较少, 底层扇入到公共的实用模块中去(底层模块有高扇入)。

4. 模块的作用域应该在控制域之内

模块的作用域定义为受该模块内一个判定影响的所有模块的集合。模块的控制域是这个模块本身以及所有直接或间接从属于它的模块的集合。

模块 C 的控制范围: C、D、E、F、G、H, 如果模块 C 作出的决策影响了模块 L, L 超出了 C 的控制范围。(难于理解; 会出现控制耦合)

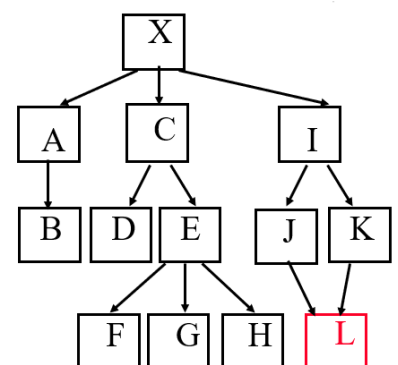
1、把做决策的点 C 上移

2、把 L 移到 C 的控制域中

在一个设计得好的系统中,

所有受判定影响的模块应该从属于做出判定的那个模块,

最好局限于做出判定的那个模块本身及它的直属下级模块。



5. 力争降低模块接口的复杂程度

模块接口复杂是软件发生错误的一个主要原因。应该仔细设计模块接口, 使得信息传递简单并且和模块的功能一致。

接口复杂或不一致(即看起来传递的数据之间没有联系), 是紧耦合或低内聚的征兆, 应该重新分析这个模块的独立性。

6. 设计单入口单出口的模块

这条启发式规则警告软件工程师**不要使模块间出现内容耦合**。当从顶部进入模块并且从底部退出来时，软件是比较容易理解的，因此也是比较容易维护的。

7. 模块功能应该可以预测

模块的功能应该能够预测，但也要防止模块功能过分局限。

如果一个模块可以当做一个黑盒子，也就是说，**只要输入的数据相同就产生同样的输出**，这个模块的功能就是可以预测的。带有内部“存储器”的模块的功能可能是不可预测的，因为它的输出可能取决于内部存储器(例如某个标记)的状态。由于内部存储器对于上级模块而言是不可见的，所以这样的模块既不易理解又难于测试和维护。